

How a result gets its score

The full blend used by the `search-ranking` and `search-ranking-optimizer` packages: Elasticsearch relevance, weighted business signals, and the entropy modifier that shifts the balance between them per query.

THE MASTER FORMULA

One blend, two terms

Every hit's final score is a convex combination of a normalized text-relevance term and a weighted sum of business signals. Nothing else feeds the score.

$$\text{finalScore} = \alpha \cdot \frac{\text{score}}{\text{score} + k} + (1 - \alpha) \cdot \sum_{i=1}^n w_i \cdot s_i$$

α `relevanceWeight` — how much weight text relevance gets vs. signals. Default **0.75**. Entropy (below) shifts this per query.

score raw Elasticsearch `_score` for the hit (BM25 + any query-time boosts already applied upstream)

k `relevanceSaturationPoint` — default **12.0**. The `score/(score+k)` curve reaches exactly 0.5 when `score = k`.

w_i normalized weight of signal *i*, with $\sum w_i = 1$

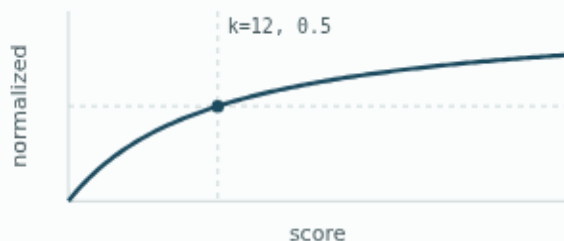
s_i normalized value of signal *i* (e.g. click-through rate, conversion rate) — see normalization, below

TERM ONE

The relevance term

Elasticsearch's unbounded `_score` is squashed into $[0, 1)$ with the same saturation shape BM25 itself uses for term frequency — here applied to the whole document score instead.

score / (score + k)



At $\text{score} = k = 12$, the curve sits at exactly 0.5 — the inflection point admin's tune the whole system around.

Where k comes from

k is a search-infra tuning constant, not a business knob — chosen from this shop's observed `_score` range (roughly 4–20).

The search-ranking-optimizer's `StatisticsCalculator` can suggest a value: it pools scores from a calibration sample and reports mean / median / P25 / P75 — the mean is the usual starting point for k.

TERM TWO

The signal term

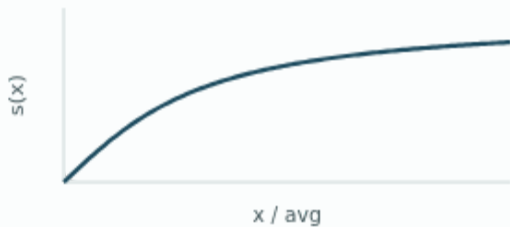
A straight weighted sum of business metrics, each normalized to $[0, 1]$ and each carrying a weight that's force-renormalized at publish time so the term always stays a true convex combination.

$$w_i = \frac{\text{enteredWeight}_i}{\sum_j \text{enteredWeight}_j}$$

Zed only enforces $\text{weight} \geq 0$ with no ceiling, so this normalization runs once at publish time, not inside the query.

signal normalization $s_i(x)$

examples in use



Admin-authored expression per metric, evaluated over x (raw value), min, max, avg, count. Result always clamped to $[1e-6, 1.0]$.

$\text{atan}(x/\text{avg})/(\pi()/2)$

saturating –
diminishing
returns past
the average

x/max

linear –
straight
share of the
ceiling
value

$\text{random}()$

tie-breaker
–
deliberately
noisy

THE ENTROPY MODIFIER

Letting the query decide how much to trust the signals

Entropy never touches the formula's shape — it only moves α before the master formula runs. A confident, single-answer query (low entropy) leans harder on text relevance; an ambiguous, flat-scoring query (high entropy) leans harder on business signals. Opt-in, off by default.

1

Probe

top N=10 raw _scores, source disabled – one lightweight extra query



2

Shannon entropy

$$p_i = \text{score}_i / \sum \text{scores}$$

$$H = -\sum p_i \ln(p_i)$$



3

Normalize

$H_{\text{norm}} = H / \ln(N) \in [0,1]$



4

Shape

$d = 1 - 2 \cdot H_{\text{norm}}$
 $\text{shift} = m \cdot \text{sign}(d) \cdot |d|^\gamma$



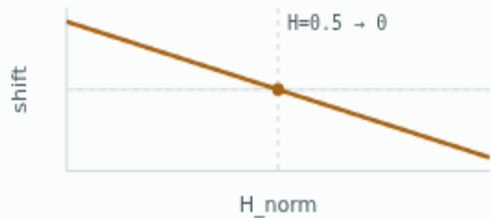
5

Apply

$\alpha = \text{clamp}(\alpha_{\text{cfg}} + \text{shift}, 0, 1)$

Any exception or empty probe result falls back unchanged to the configured α — entropy weighting can only ever be fail-safe, never break a real search.

shift vs. H_{norm} ($\gamma=1$, $m=0.25$)



$\gamma = \text{entropyWeightExponent}$ reshapes the curve away from linear if set >1 ; the shipped default is 1.0.

worked examples

QUERY SHAPE	H_{NORM}	SHIFT	α EFFECTIVE
one dominant hit	0.00	+0.25	1.00 – relevance only
typical spread	0.50	0.00	0.75 – unchanged
flat / ambiguous	1.00	−0.25	0.50 – signals half the say

REFERENCE

Every knob, in one place

SETTING KEY	SYMBOL	DEFAULT	
relevance_weight	α	0.75	relevance vs. signal blend core
relevance_saturation_point	k	12.0	where the relevance curve hits 0.5 core
spy_search_ranking_metric.weight	w_i	$\Sigma=1$	per-metric weight, normalized at publish core
entropy_probe_result_size	N	10	how many top hits the entropy probe samples entropy
entropy_weight_exponent	γ	1.0	shape of the shift curve; 1.0 = linear entropy
entropy_weight_shift_magnitude	m	0.25	max $\pm$ shift entropy can apply to α entropy
isEntropyWeightingEnabled()	—	false	code-level flag; off unless explicitly enabled entropy

The same numbers, rendered live for a single hit, show up in the search-debug overlay's score breakdown — it reads the identical `relevanceWeight · normalizedRelevance + (1-relevanceWeight) · signalTotal` by construction, not a re-derivation.

■ core formula
 ■ entropy modifier

search-ranking · search-ranking-optimizer